



4.2. All Pairs Shortest Path

"Connecting every node, mapping every route."

Main Section:

4. Dynamic Programming

Contents:

[1. Problem Definition](#)

[2. Workflow](#)

[Summary](#)

[References](#)

Imagine standing in the middle of a massive transportation network. Every city connects to another city through roads, flights, railways, or communication links. Now imagine asking not just: ***"How do I get from A to B?"***

But instead: ***"What is the shortest possible route between every pair of locations in the entire network?"***. That is the essence of the **All Pairs Shortest Path problem**.

Unlike simpler pathfinding problems that focus on a single starting point, this problem asks us to understand the entire graph at once. Every node becomes both a starting point and a destination. Every possible relationship between vertices matters.

This is where the famous **Floyd-Warshall algorithm** comes in, created by Robert Floyd and Stephen Warshall. The algorithm systematically explores whether traveling through an intermediate node creates a shorter route between two vertices. Instead of greedily committing to one path immediately, it continuously refines the distances between every pair of nodes until no shorter path can exist anymore.

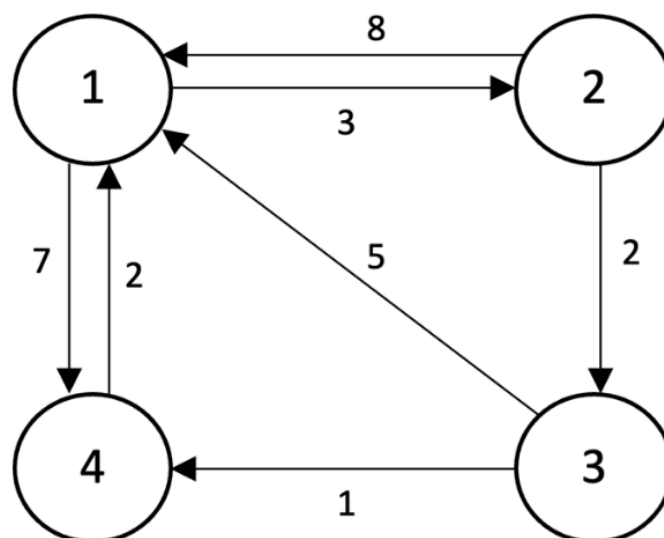
What makes this algorithm especially powerful is its versatility. It works on dense graphs, directed or undirected graphs, positive edge weights, even negative edge weights (as long as there are no negative cycles)

Because of this, Floyd–Warshall appears in many real-world systems: GPS and navigation systems; Network routing; Airline optimization; Social network analysis; Communication infrastructures.

And unlike many graph algorithms that focus on one source at a time, Floyd–Warshall views the graph globally. It attempts to understand the entire connectivity structure of the network simultaneously.

1. Problem Definition

Consider the weighted graph below.



Suppose vertex 1 is chosen as the starting point. We may want to find:

- The shortest path from $1 \rightarrow 2$
- The shortest path from $1 \rightarrow 3$
- The shortest path from $1 \rightarrow 4$

At first glance, this resembles the classic **Single Source Shortest Path** problem, commonly solved using Edsger W. Dijkstra's famous Dijkstra Algorithm.

And technically, yes, we *could* solve the problem that way. We could:

1. Run Dijkstra starting from vertex 1
2. Then run it again from vertex 2
3. Then again from vertex 3
4. Continue until every vertex has been used as the source
5. Eventually, we would obtain the shortest distance between every pair of vertices.

But here is where the issue begins. For small graphs, repeatedly running Dijkstra is perfectly acceptable. But as the graph becomes larger and denser, this approach quickly becomes computationally expensive.

Imagine thousands of cities, millions of network connections, massive transportation systems, and large-scale internet routing graphs. Running a shortest path algorithm over and over again for every single node becomes increasingly inefficient.

And there is another important limitation. Traditional greedy approaches like Dijkstra's algorithm work best when edge weights are non-negative. Once negative weights appear, things become far more complicated.

So instead of solving the problem repeatedly from multiple starting points, we need a method that can analyze the graph as a whole. A method that gradually improves every possible path simultaneously. That is exactly the problem Floyd–Warshall was designed to solve.

2. Workflow

Now we will finally solve the graph using the **Floyd–Warshall Algorithm**. Unlike Greedy approaches that attempt to immediately commit to a locally optimal path, Floyd–Warshall takes a much more patient approach. It systematically examines *every possible route* between every pair of vertices.

The idea is deceptively simple. Suppose we want to find the shortest path from vertex X to vertex Y . There may exist: A direct path from $X \rightarrow Y$, or an indirect path that travels through several intermediate vertices before reaching Y

And here is the important realization: The direct edge is not always the shortest path. Sometimes, taking a longer-looking route through intermediate vertices

produces a smaller total cost.

This is exactly why the algorithm repeatedly asks the same question:

"If we allow another vertex to become an intermediate stop... can we obtain a shorter path?"

Instead of solving the graph all at once, Floyd–Warshall gradually improves the solution step-by-step using tables.

Initial Table — (A^0)

We first construct the initial adjacency matrix, denoted as:

$$A^0 = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$$

This table only contains the **direct edges**. For example:

- The cost from $1 \rightarrow 2$ is 3
- The cost from $2 \rightarrow 1$ is 8
- The cost from $3 \rightarrow 4$ is 1

The diagonal values remain 0 because the distance from a vertex to itself is always zero. If no direct edge exists between two vertices, we place: ∞ , which represents an unreachable path. For instance: There is no direct edge from $1 \rightarrow 3$. Therefore, its cost is initially **infinity (∞)**

Constructing (A^1)

Now we allow **vertex 1** to become an intermediate vertex. This means we examine whether traveling through vertex 1 produces shorter paths.

The update rule becomes:

$$A^k[i, j] = \min (A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$$

Let us examine one example carefully. Suppose we want to determine whether the path from $2 \rightarrow 3$ becomes shorter if we travel through vertex 1. The direct

path is: $A^0[2, 3] = 2$. The indirect path through vertex 1 becomes: $A^0[2, 1] + A^0[1, 3] = 8 + \infty$

$\Rightarrow$ Since: $2 < 8 + \infty$, the direct path remains shorter. So the distance from $2 \rightarrow 3$ stays 2.

However, some paths *do* improve. For example: There is no direct path from $2 \rightarrow 4$. But we can travel: $2 \rightarrow 1 \rightarrow 4$, with total cost: $8 + 7 = 15$. So infinity is replaced by 15.

After considering all possible paths through vertex 1, we obtain:

$$A^1 = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix}$$

Constructing (A^2)

Next, we allow **vertex 2** to become the intermediate vertex. Now the algorithm checks whether traveling through 2 creates shorter routes. Some paths improve significantly.

- For example: $1 \rightarrow 2 \rightarrow 3$, has cost: $3 + 2 = 5$, which is shorter than infinity. So the path from $1 \rightarrow 3$ is updated to 5.

The updated table becomes:

$$A^2 = \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

Constructing (A^3)

Now we use **vertex 3** as the intermediate vertex. This stage creates several important improvements.

- For instance: $2 \rightarrow 3 \rightarrow 4$ has cost: $2 + 1 = 3$, which is much smaller than 15. So: $A^3[2, 4] = 3$.

- Similarly: $2 \rightarrow 3 \rightarrow 1$ has cost: $2 + 5 = 7$, which improves the previous cost of 8. After considering all paths through vertex 3, we get:

$$A^3 = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

Constructing (A^4)

Finally, we allow **vertex 4** to become the intermediate vertex. Again, the algorithm checks whether traveling through 4 improves any remaining paths.

- For example: $3 \rightarrow 4 \rightarrow 1$ has cost: $1 + 2 = 3$, which is shorter than 5. So the path from $3 \rightarrow 1$ improves to 3.
- Likewise: $3 \rightarrow 4 \rightarrow 2$ has cost: $1 + 5 = 6$, which improves the previous cost of 8.

The final shortest path table becomes:

$$A^4 = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

At this stage, the algorithm has completely examined every possible indirect route between every pair of vertices. And now something interesting becomes visible:

- Some direct edges remained the shortest paths
- Some indirect paths became shorter than the direct ones
- Some unreachable paths (∞) eventually became reachable through intermediate vertices

This is the beauty of Floyd–Warshall. Instead of greedily making decisions immediately, it continuously improves the solution by reusing previously solved sub-problems.

That is precisely why this problem is considered one of the most elegant applications of **Dynamic Programming**. The solution naturally possesses

optimal substructure, repeated overlapping computations, progressive refinement of solutions.

The shortest path between two vertices can be built from previously computed shortest paths between smaller vertex combinations. And through this repeated refinement process, the algorithm eventually discovers the globally optimal solution for every pair of vertices simultaneously.

Below is the simple Python implementation of Knapsack Problem:

```
def floyd_warshall_algorithm(graph):
    n = len(graph)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                # Check if there exists shorter path in vertex k
                if graph[i][j] > graph[i][k] + graph[k][j]:
                    graph[i][j] = graph[i][k] + graph[k][j]
    return graph
```

```
# Initialize the table A0
infinity = float('inf')
A = np.array([[0, 3, infinity, 7],
              [8, 0, 2, infinity],
              [5, infinity, 0, 1],
              [2, infinity, infinity, 0]])
solution = floyd_warshall_algorithm(A)
print(solution)
```

```
[[0. 3. 5. 6.]
 [5. 0. 2. 3.]
 [3. 6. 0. 1.]
 [2. 5. 7. 0.]]
```

Summary

The **All Pairs Shortest Path** problem is one of the most important problems in graph theory.

Technically, it can be solved using several approaches. Repeatedly applying Dijkstra's algorithm is one possibility. However, as graphs become larger and more complex, repeatedly solving single-source problems becomes increasingly inefficient.

This is where the Floyd–Warshall algorithm becomes powerful. Instead of focusing on one source at a time, it examines the graph globally. By gradually allowing more intermediate vertices, the algorithm continuously improves previously discovered paths until every shortest path is found.

And more importantly, this problem perfectly demonstrates the philosophy of Dynamic Programming: ***Break a large optimization problem into smaller sub-problems → solve them progressively → reuse their solutions to construct the final optimal answer.***

That is why Floyd–Warshall remains one of the most elegant and classic Dynamic Programming algorithms ever created.

References

<https://www.geeksforgeeks.org/dsa/floyd-warshall-algorithm-dp-16/>

<https://www.youtube.com/watch?v=oNI0rf2P9gE&t=405s>

<https://www.youtube.com/watch?v=4NQ3HnhyNfQ>